

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 - SORTING



NAMA : Irfan Ali Wicaksono

NIM : 109082500133

KELAS S1IF-13-05

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO

A. Dasar Teori

Pada praktikum ini digunakan konsep **array** dan **algoritma sorting**. Array merupakan struktur data yang digunakan untuk menyimpan sekumpulan data dengan tipe yang sama dalam satu variabel. Setiap elemen pada array memiliki indeks yang digunakan untuk mengakses data tersebut.

Sorting adalah proses mengurutkan data berdasarkan urutan tertentu, seperti dari nilai terkecil ke terbesar (ascending) atau dari terbesar ke terkecil (descending). Pada praktikum ini digunakan dua metode sorting, yaitu **Selection Sort** dan **Insertion Sort**.

Selection Sort bekerja dengan mencari elemen terkecil atau terbesar dari bagian data yang belum terurut, kemudian menukarnya dengan posisi yang sesuai. Sedangkan Insertion Sort bekerja dengan menyisipkan setiap elemen ke posisi yang tepat pada bagian data yang sudah terurut sehingga data menjadi terurut secara bertahap. Kedua metode ini digunakan untuk membantu pengolahan dan penyusunan data agar lebih mudah dianalisis.

B. Guided

1. Contoh Guided 1

“Program Pengurutan Data Array Menggunakan Metode Selection Sort”

```
package main
import "fmt"

func SelectionSortArray(arr *[8]int) {
    var idx_min, i, j int
    for i = 0; i < len(arr)-1; i++ { //perulangan luar
        idx_min = i;
        for j = i + 1; j < len(arr); j++ { //perulangan
dalam
            if arr[j] < arr[idx_min] { //kondisional
                idx_min = j
            }
        }
        if idx_min != i { //swap
            arr[i], arr[idx_min] = arr[idx_min], arr[i]
        }
    }
}

func main() {
    var arrAngka [8]int

    for i := 0; i < len(arrAngka); i++ {
```

```

        fmt.Printf("Masukkan data angka indeks ke-%d:
", i)
        fmt.Scan(&arrAngka[i])
    }
    fmt.Println()

    fmt.Println("===SEBELUM DISORTING===")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " ")
    }
    fmt.Println()

    SelectionSortArray(&arrAngka)
    fmt.Println("===SESUDAH DISORTING===")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " ")
    }
}

```

Output :

```

Masukkan data angka indeks ke-0: 8
Masukkan data angka indeks ke-1: 3
Masukkan data angka indeks ke-2: 7
Masukkan data angka indeks ke-3: 1
Masukkan data angka indeks ke-4: 5
Masukkan data angka indeks ke-5: 2
Masukkan data angka indeks ke-6: 9
Masukkan data angka indeks ke-7: 4

===SEBELUM DISORTING===
8 3 7 1 5 2 9 4
===SESUDAH DISORTING===
1 2 3 4 5 7 8 9

```

2. Contoh Guided 2

“Program Pengurutan Data Mahasiswa Berdasarkan IPK Menggunakan Metode Selection Sort”

```
package main
import "fmt"

type mahasiswa struct {
    nama, nim string
    IPK float64
}

func SelectionSortArray(sMHS *[5]mahasiswa) {
    var idx_min, i, j int
    for i = 0; i < len(sMHS)-1; i++ { //perulangan luar
        idx_min = i
        for j = i + 1; j < len(sMHS); j++ { //loop dalam
            mencari nilai eksrem
            if sMHS[j].IPK < sMHS[idx_min].IPK { //soring
                erkecil ke erbesar
                idx_min = j
            }
        }
        if idx_min != i { //swap
            sMHS[i], sMHS[idx_min] = sMHS[idx_min], sMHS[i]
        }
    }
}

func main() {
    var structMHS [5]mahasiswa

    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("Masukkan data mahasiswa indeks ke-
%d:\n", i)
        fmt.Print("Nama: ")
        fmt.Scan(&structMHS[i].nama)
        fmt.Print("NIM: ")
        fmt.Scan(&structMHS[i].nim)
        fmt.Print("IPK: ")
        fmt.Scan(&structMHS[i].IPK)
    }
    fmt.Println()

    fmt.Println("===SEBELUM DISORTING===")
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("Data mahasiswa indeks ke-%d:\n", i)
        fmt.Printf("Nama: %s\n", structMHS[i].nama)
        fmt.Printf("NIM: %s\n", structMHS[i].nim)
```

```
        fmt.Printf("IPK: %.2f\n", structMHS[i].IPK)
    }
    fmt.Println("=====")
    fmt.Println()

    SelectionSortArray(&structMHS)
    fmt.Println("===SESUDAH DISORTING===")
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("Data mahasiswa indeks ke-%d:\n", i)
        fmt.Printf("Nama: %s\n", structMHS[i].nama)
        fmt.Printf("NIM: %s\n", structMHS[i].nim)
        fmt.Printf("IPK: %.2f\n", structMHS[i].IPK)
    }
    fmt.Println("=====")
    fmt.Println()
}
```

Output :

```

===SEBELUM DISORTING===
Data mahasiswa indeks ke-0:
Nama: rozi
NIM: 638358036
IPK: 3.50
Data mahasiswa indeks ke-1:
Nama: irfan
NIM: 1974850
IPK: 3.70
Data mahasiswa indeks ke-2:
Nama: rafly
NIM: 109082500112
IPK: 3.80
Data mahasiswa indeks ke-3:
Nama: zidan
NIM: 109082500113
IPK: 3.85
Data mahasiswa indeks ke-4:
Nama: galih
NIM: 109082500114
IPK: 3.60
=====

===SESUDAH DISORTING===
Data mahasiswa indeks ke-0:
Nama: rozi
NIM: 638358036
IPK: 3.50
Data mahasiswa indeks ke-1:
Nama: galih
NIM: 109082500114
IPK: 3.60
Data mahasiswa indeks ke-2:
Nama: irfan
NIM: 1974850
IPK: 3.70
Data mahasiswa indeks ke-3:
Nama: rafly
NIM: 109082500112
IPK: 3.80
Data mahasiswa indeks ke-4:
Nama: zidan
NIM: 109082500113

```

3. Contoh Guided 3

“Program Pengurutan Data Mahasiswa Berdasarkan Nama Menggunakan Metode Insertion Sort”

```

package main
import "fmt"

type mahasiswa struct {
    nama, nim string
    IPK float64
}

```

```

type arrMhs [100]mahasiswa

func insertionSortStruct(T *arrMhs, n int) {
    var temp mahasiswa
    var i, j int

    for i = 1; i < n; i++ {
        temp = T[i]
        j = i - 1
        for j >= 0 && T[j].nama > temp.nama {
            T[j+1] = T[j]
            j = j - 1
        }
        T[j+1] = temp
    }
}

func main() {
    var data arrMhs
    var n, i int

    fmt.Print("Masukkan jumlah mahasiswa: ")
    fmt.Scan(&n)

    for i = 0; i < n; i++ {
        fmt.Printf("Masukkan data mahasiswa indeks ke-
%d:\n", i)

        fmt.Print("Nama: ")
        fmt.Scan(&data[i].nama)

        fmt.Print("NIM: ")
        fmt.Scan(&data[i].nim)

        fmt.Print("IPK: ")
        fmt.Scan(&data[i].IPK)
    }
    fmt.Println("\nData mahasiswa sebelum disorting:")
    for i = 0; i < n; i++ {
        fmt.Printf("Nama: %s, NIM: %s, IPK: %.2f\n",
data[i].nama, data[i].nim, data[i].IPK)
    }

    insertionSortStruct(&data, n)

    fmt.Println("\nData mahasiswa setelah disorting:")
    for i = 0; i < n; i++ {
        fmt.Printf("Nama: %s, NIM: %s, IPK: %.2f\n",
data[i].nama, data[i].nim, data[i].IPK)
    }
}

```

Output :

```
Masukkan jumlah mahasiswa: 4
Masukkan data mahasiswa indeks ke-0:
Nama: irfan
NIM: 109082500133
IPK: 3.70
Masukkan data mahasiswa indeks ke-1:
Nama: rozi
NIM: 109082500134
IPK: 3.60
Masukkan data mahasiswa indeks ke-2:
Nama: rafly
NIM: 109082500135
IPK: 3.75
Masukkan data mahasiswa indeks ke-3:
Nama: zidan
NIM: 109082500136
IPK: 3.80

Data mahasiswa sebelum disorting:
Nama: irfan, NIM: 109082500133, IPK: 3.70
Nama: rozi, NIM: 109082500134, IPK: 3.60
Nama: rafly, NIM: 109082500135, IPK: 3.75
Nama: zidan, NIM: 109082500136, IPK: 3.80

Data mahasiswa setelah disorting:
Nama: irfan, NIM: 109082500133, IPK: 3.70
Nama: rafly, NIM: 109082500135, IPK: 3.75
Nama: rozi, NIM: 109082500134, IPK: 3.60
Nama: zidan, NIM: 109082500136, IPK: 3.80
```


C. Unguided

1. Soal Unguided 1

- 1) Hercules, preman terkenal seantero ibukota, memiliki kerabat di banyak daerah. Tentunya Hercules sangat suka mengunjungi semua kerabatnya itu.

Diberikan masukan nomor rumah dari semua kerabatnya di suatu daerah, buatlah program **rumahkerabat** yang akan menyusun nomor-nomor rumah kerabatnya secara terurut membesar menggunakan algoritma selection sort.

Jawaban :

```
package main

import "fmt"

const NMAX = 100000

var rumah [NMAX]int

func selectionSort(m int) {
    for i := 0; i < m-1; i++ {
        min := i

        for j := i + 1; j < m; j++ {
            if rumah[j] < rumah[min] {
                min = j
            }
        }

        rumah[i], rumah[min] = rumah[min], rumah[i]
    }
}

func main() {
    var n, m int

    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Scan(&m)

        for j := 0; j < m; j++ {
            fmt.Scan(&rumah[j])
        }

        selectionSort(m)

        for j := 0; j < m; j++ {
```

```

        fmt.Print(rumah[j], " ")
    }
    fmt.Println()
}

```

Output :

```

3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 4 9

```

Penjelasan :

Program ini membaca data nomor rumah kerabat Hercules pada setiap daerah, kemudian mengurutkannya dari nomor terkecil hingga terbesar menggunakan algoritma Selection Sort. Algoritma ini bekerja dengan mencari nilai terkecil dari data yang belum terurut, lalu menukarnya ke posisi yang sesuai. Setelah seluruh data terurut, program menampilkan nomor rumah yang sudah tersusun secara menaik untuk setiap daerah.

2. Soal Unguided 2

- 2) Belakang diketahui ternyata Hercules itu tidak berani menyeberang jalan, maka selalu diusahakan agar hanya menyeberang jalan sesedikit mungkin, hanya diujung jalan. Karena nomor rumah sisi kiri jalan selalu ganjil dan sisi kanan jalan selalu genap, maka buatlah program **kerabat dekat** yang akan menampilkan nomor rumah mulai dari nomor yang ganjil lebih dulu terurut membesar dan kemudian menampilkan nomor rumah dengan nomor genap terurut mengecil.

Jawaban :

```

package main

import "fmt"

```

```

const NMAX = 100000

func main() {
    var n, m, x int

    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        var ganjil, genap [NMAX]int
        jmlGanjil := 0
        jmlGenap := 0

        fmt.Scan(&m)

        for j := 0; j < m; j++ {
            fmt.Scan(&x)

            if x%2 == 0 {
                genap[jmlGenap] = x
                jmlGenap++
            } else {
                ganjil[jmlGanjil] = x
                jmlGanjil++
            }
        }

        // Selection Sort ganjil naik
        for j := 0; j < jmlGanjil-1; j++ {
            min := j
            for k := j + 1; k < jmlGanjil; k++ {
                if ganjil[k] < ganjil[min] {
                    min = k
                }
            }
            ganjil[j], ganjil[min] = ganjil[min],
ganjil[j]
        }

        // Selection Sort genap turun
        for j := 0; j < jmlGenap-1; j++ {
            max := j
            for k := j + 1; k < jmlGenap; k++ {
                if genap[k] > genap[max] {
                    max = k
                }
            }
            genap[j], genap[max] = genap[max], genap[j]
        }

        for j := 0; j < jmlGanjil; j++ {

```

```

        fmt.Print(ganjil[j], " ")
    }

    for j := 0; j < jmlGenap; j++ {
        fmt.Print(genap[j], " ")
    }

    fmt.Println()
}
}

```

Output :

```

3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4
PS C:\Users\lenovo\OneDr
\UnGuided-no2\no2.go"
1
5 12 1 13 2 8
1 13 12 8 2

```

Penjelasan :

Program ini membaca nomor rumah kerabat pada setiap daerah, kemudian memisahkan bilangan ganjil dan genap ke dalam dua array. Bilangan ganjil diurutkan secara menaik, sedangkan bilangan genap diurutkan secara menurun menggunakan algoritma Selection Sort. Setelah itu, program menampilkan semua bilangan ganjil terlebih dahulu, lalu diikuti oleh bilangan genap.

3. Soal Unguided 3

- 1) Buatlah sebuah program yang digunakan untuk membaca data integer seperti contoh yang diberikan di bawah ini, kemudian diurutkan (menggunakan metoda *insertion sort*), dan memeriksa apakah data yang terurut berjarak sama terhadap data sebelumnya.

Masukan terdiri dari sekumpulan bilangan bulat yang diakhiri oleh bilangan negatif. Hanya bilangan non negatif saja yang disimpan ke dalam array.

Jawaban :

```
package main

import "fmt"

const NMAX = 100

func main() {
    var data [NMAX]int
    var n, x int

    fmt.Scan(&x)
    for x >= 0 {
        data[n] = x
        n++
        fmt.Scan(&x)
    }

    // Insertion Sort
    for i := 1; i < n; i++ {
        temp := data[i]
        j := i - 1

        for j >= 0 && data[j] > temp {
            data[j+1] = data[j]
            j--
        }

        data[j+1] = temp
    }

    // Cetak array yang sudah terurut
    for i := 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }
    fmt.Println()

    // Cek jarak
```

```

        jarak := data[1] - data[0]
        tetap := true

        for i := 2; i < n; i++ {
            if data[i]-data[i-1] != jarak {
                tetap = false
            }
        }

        if tetap {
            fmt.Println("Data berjarak", jarak)
        } else {
            fmt.Println("Data berjarak tidak tetap")
        }
    }
}

```

Output :

```

\UnGuided-no3\no3.go"
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6
PS C:\Users\lenovo\OneDrive\
\UnGuided-no3\no3.go"
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 26 32 38 40
Data berjarak tidak tetap

```

Penjelasan :

Program ini berhasil menerapkan algoritma Insertion Sort pada array struktur mahasiswa untuk mengurutkan data berdasarkan nama secara ascending (A–Z). Dengan menggunakan metode ini, data mahasiswa menjadi lebih teratur dan mudah untuk dicari maupun dikelola.

4. Soal Unguided 4

- 2) Sebuah program perpustakaan digunakan untuk mengelola data buku di dalam suatu perpustakaan. Misalnya terdefinisi struct dan array seperti berikut ini:

```
const nMax : integer = 7919
type Buku = <
    id, judul, penulis, penerbit : string
    eksemplar, tahun, rating : integer >

type DaftarBuku = array [ 1..nMax] of Buku
Pustaka : DaftarBuku
nPustaka: integer
```

```
package main

import "fmt"

const NMAX = 719

type buku struct {
    id      int
    judul   string
    penulis string
    rating  int
}

type DaftarBuku [NMAX]buku

func BuatBuku(pustaka *DaftarBuku, n int) {
    for i := 0; i < n; i++ {
        fmt.Scan(
            &pustaka[i].id,
            &pustaka[i].judul,
            &pustaka[i].penulis,
            &pustaka[i].rating,
        )
    }
}

func CetakFavorit(pustaka DaftarBuku, n int) int {
    idx := 0

    for i := 1; i < n; i++ {
        if pustaka[i].rating > pustaka[idx].rating {
            idx = i
        }
    }
}
```

```

        return idx
    }

    func TutupBuku(pustaka DaftarBuku, n int, x int) int {
        for i := 0; i < n; i++ {
            if pustaka[i].id == x {
                return i
            }
        }
        return -1
    }

    func CariBuku(pustaka DaftarBuku, n int, x string) int
    {
        for i := 0; i < n; i++ {
            if pustaka[i].judul == x {
                return pustaka[i].rating
            }
        }
        return -1
    }

    func main() {
        var pustaka DaftarBuku
        var n int
        var idCari int
        var judulCari string

        fmt.Scan(&n)

        BuatBuku(&pustaka, n)

        // Buku favorit
        idx := CetakFavorit(pustaka, n)
        fmt.Println("Buku favorit:", pustaka[idx].judul)

        // Cari berdasarkan rating tertinggi
        fmt.Scan(&judulCari)
        rating := CariBuku(pustaka, n, judulCari)
        fmt.Println("Rating:", rating)

        // Cari berdasarkan ID
        fmt.Scan(&idCari)
        idx = TutupBuku(pustaka, n, idCari)

        if idx != -1 {
            fmt.Println("Ditemukan:",
pustaka[idx].judul)
        } else {
            fmt.Println("Buku tidak ditemukan")
        }
    }

```



```
}
```

Output :

```
3
101 LaskarPelangi AndreaHirata 95
102 Bumi TereLiye 90
103 AtomicHabits JamesClear 98
Buku favorit: AtomicHabits
Bumi
Rating: 90
102
Ditemukan: Bumi
```

Penjelasan :

Program ini digunakan untuk mengelola data buku dalam sebuah perpustakaan. Data setiap buku terdiri dari ID, judul, penulis, dan rating. Program terlebih dahulu membaca dan menyimpan seluruh data buku ke dalam array. Setelah itu, program mencari buku favorit berdasarkan rating tertinggi, mencari rating suatu buku berdasarkan judul yang diberikan, serta mencari buku berdasarkan ID tertentu. Jika buku ditemukan, program akan menampilkan informasi yang sesuai, sedangkan jika tidak ditemukan akan menampilkan pesan bahwa buku tidak ditemukan.

Kesimpulan:

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma Selection Sort dan Insertion Sort dapat digunakan untuk mengurutkan data dalam array sesuai kebutuhan. Selection Sort mengurutkan data dengan memilih elemen terkecil atau terbesar secara bertahap, sedangkan Insertion Sort mengurutkan data dengan menyisipkan elemen pada posisi yang tepat. Dengan menerapkan kedua algoritma tersebut, data menjadi lebih terstruktur, mudah dicari, dan lebih mudah diolah. Selain itu, praktikum ini juga membantu memahami penggunaan array, perulangan, percabangan, prosedur, fungsi, dan struktur data dalam pemrograman.